AI4101 · ARTIFICIAL INTELLIGENCE PRINCIPLES · LAB 2

From Search to Routes

Ekram as a Travelling Salesman Problem

Work	In pairs, in the lab. One report per pair.
Time	One lab block (4 hours).
Tools	Google Colab in the browser. No installation. A Google account.
Files	Notebook `AI4101_M3_Lab2_Ekram_TSP_Search.ipynb` and report template `AI4101_M3_Lab2_Ekram_TSP_Search_ReportTemplate.docx`, both on the lab page `phdahmad.github.io/AI4101/labs/lab2`.
Submit	The report (PDF), one week after the lab, the way your instructor announces in class.
Next	Assignment 2 builds on this lab. Keep your notebook.
Links to the course	Module 3 (sections 3.1 to 3.3) and Module 2 (sections 2.3 to 2.5).

> **About the Ekram scenario**
>
> The scenario in this lab is **simulated**. It is inspired by the work of Ekram Al-Jood Association in Makkah. The warehouse, the donors and all numbers are invented for learning.

What you will learn

- Turn a real task into a **search problem** with the five parts of Module 3.
- Build a **distance matrix** and compute the length of a route.
- Find the **optimal** route by trying every order, and a quick route with a **greedy** method.
- Measure how fast the problem grows, and see why trying every order stops working.
- Compare your result with an academic **benchmark**, and see what a real business would still need.

Words you will use

Word	Meaning in this lab
TSP	Travelling Salesman Problem: visit every place exactly once and return to the start, with the shortest total distance.
Route	One complete order of visits, starting and ending at the warehouse, for example 0 → 3 → 1 → 2 → 0.
Distance matrix	A table where row *i*, column *j* holds the distance from place *i* to place *j*.
Brute force	Try every possible route and keep the shortest. It always finds the optimal route, if you can wait.
Greedy method	Make the choice that looks best right now, and never change it later.
n! (n factorial)	1 × 2 × 3 × … × n. The number of orders of n donors. 6! = 720.
Gap	How far a route is from the optimal one: (your length − optimal length) ÷ optimal length × 100%.
Benchmark	A published problem with a known optimal answer, used to test methods fairly.

From Module 3 you already know: **state**, **action**, **transition model**, **goal test**, **action cost**, **path cost**, and **optimal solution** (the cheapest solution).

Before you start (15 min)

1. Open the lab page `phdahmad.github.io/AI4101/labs/lab2` and click **Open in Colab**.
2. Click **File → Save a copy in Drive**. Work only in your copy; the original does not keep your changes.
3. Run a cell with **Shift + Enter**. Run the cells **from top to bottom**. If something looks wrong, use **Runtime → Restart and run all**.
4. Open the report template and fill it in as you go.

> **Using AI tools in this lab**
>
> You may ask an AI assistant to explain an error or an idea. The code in the three TODO cells must be yours, and you must be able to explain every line: Assignment 2 starts with a short live check of your notebook.

Part 1 · The Ekram scenario (15 min)

One vehicle leaves the Ekram warehouse in **Al-Awali**. It visits six donors to collect surplus food, and returns to the warehouse. Each donor is visited **exactly once**.

The question: in which order should the vehicle visit the donors so that the total distance is as short as possible?

Run the first two code cells. They list the seven places and draw them on a simple map. The warehouse is the square.

Part 2 · Model the problem on paper (35 min)

Close the laptop for this part. With your partner, write the **five parts** of this search problem (Module 3, section 3.1) in section 1 of the report:

Part	A question to guide you
Initial state	Where is the vehicle at the start, and which donors has it visited?
Actions	From a given state, what can the vehicle do next?
Transition model	After an action, what is the new state? Write it as RESULT(state, action) = new state.
Goal test	How do you know the task is finished?
Action cost	What does one step cost? Then: what is the path cost of a full route?

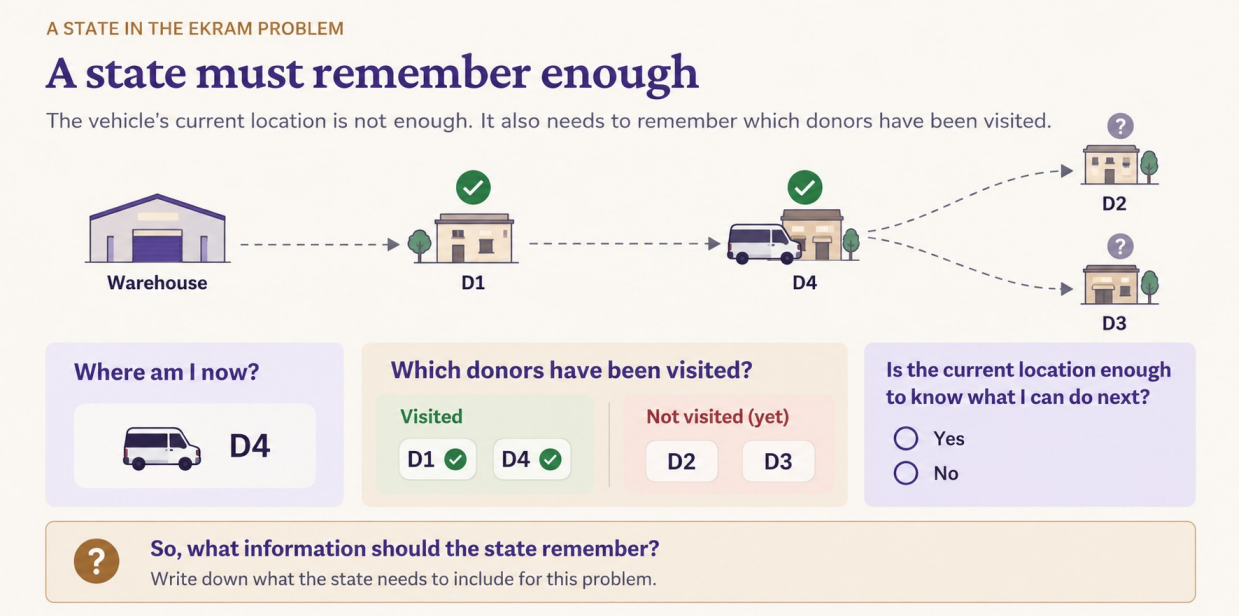


Figure 1. A state must contain enough information to determine what can happen next.

Check yourself

Is "the vehicle is in Al-Aziziyah" enough to describe a state? Think about what else the vehicle must remember so that it does not visit the same donor twice.

Part 3 · The distance matrix (20 min)

We use the **straight-line distance** ("as the crow flies") between two points on the Earth, in kilometres. The function `straight_line_km` is given.

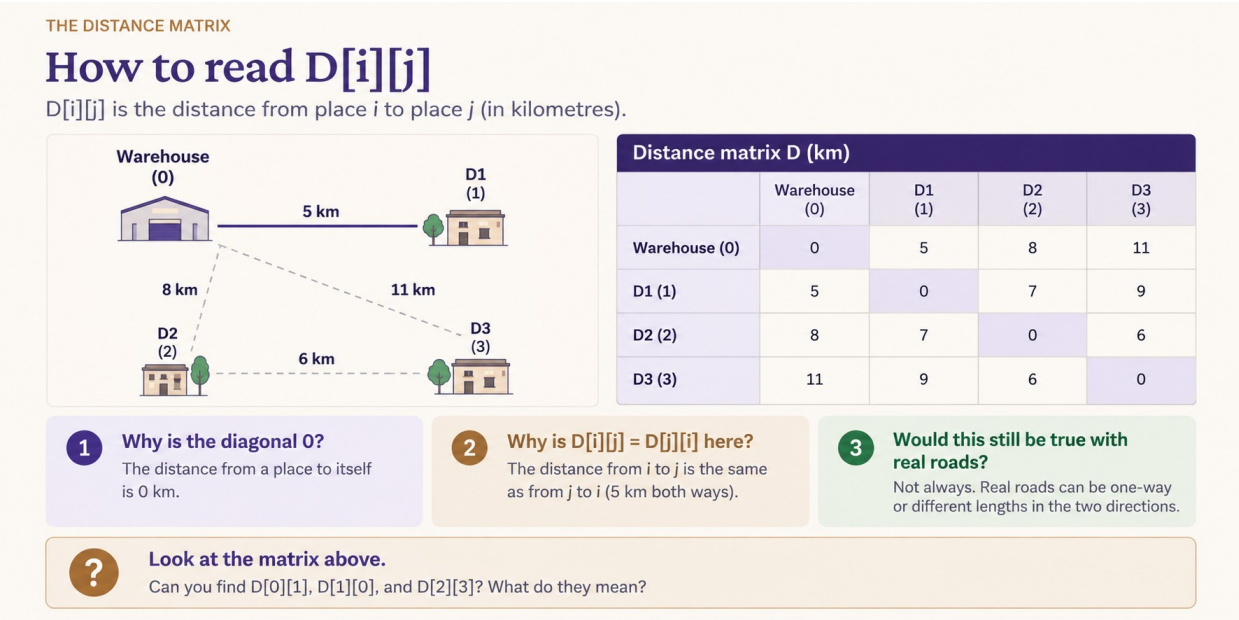


Figure 2. Reading an entry in the distance matrix.

1. **TODO 1:** complete `distance_matrix`, so that `D[i][j]` is the distance from place *i* to place *j*.
2. Run the cell. Check two things: the diagonal is 0, and `D[i][j]` equals `D[j][i]`.
3. Take a screenshot of the printed matrix for section 2 of the report.

> **Why straight lines?**
>
> Real roads are longer than straight lines, and traffic changes the time. We start simple on purpose: the method is the same, only the numbers in the matrix change. Lab 3 shows what changes with real roads.

Part 4 · Brute force: try every order (20 min)

1. **TODO 2:** complete `route_length`, which adds the distance of every step of a route.
2. Run the brute-force cell. It tries every order of the six donors and keeps the shortest route.
3. Write in section 3 of the report: the best route (with the names of the places), its length, how many routes were tried, and the time.

> **Screenshot 1 — required**
>
> The map of the optimal route. Paste it into section 3 of the report.

> **Check yourself**
>
> Brute force tried 720 routes. But the route 0 → 1 → 2 → … → 0 and the same route driven backwards have the same length. How many **different** routes are there really?

Part 5 · Nearest neighbour: a greedy method (30 min)

A quicker idea: **from where you are, always drive to the closest donor you have not visited yet**. When all donors are visited, go back to the warehouse.

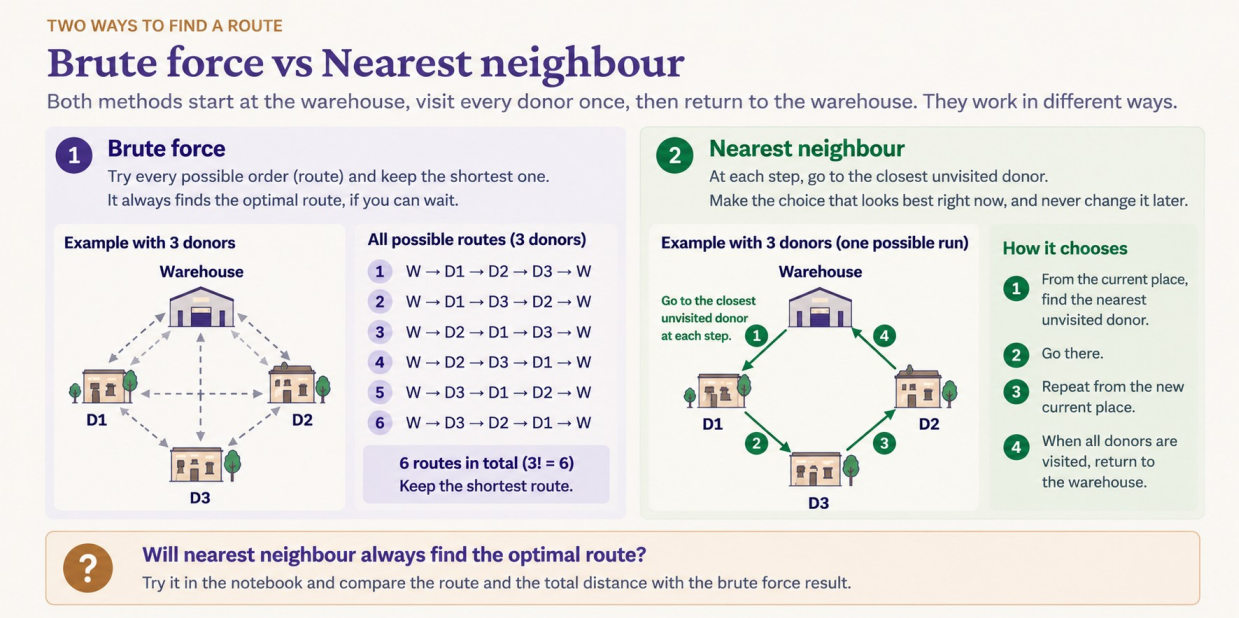


Figure 3. Two different ways to construct a route: exhaustive search and a greedy choice.

1. **TODO 3:** complete `nearest_neighbour`.
2. Run the cell. Write the route, its length and the **gap** to the optimal route in section 4 of the report.
3. Put the two maps side by side: the optimal route and the nearest-neighbour route. Where does the greedy route make its first bad choice? Is there a place where it crosses itself?

> **Greedy is not the same as optimal**
>
> Nearest neighbour is fast, and each step looks sensible. But an early choice that looks good can force an expensive step later, for example a long drive back to the warehouse at the end.

Part 6 · How fast does the problem grow? (30 min)

1. Run the cell that makes **random** problems with 4 to 10 donors and times brute force on each. The `seed` makes the random numbers the same every time, so results can be checked.
2. Copy the table (donors, routes, time) into section 5 of the report.
3. Run the next cell. It draws the growth, and **estimates** the time for 12, 15, 20 and 40 donors from your own measurements.

> **Screenshot 2 — required**
>
> The growth chart. Paste it into section 5 of the report, with the four estimates.

Your times will differ from other pairs: they depend on the computer. The **shape** is what matters. Each extra donor multiplies the number of routes by the new number of donors.

Part 7 · An academic benchmark: eil51 (25 min)

Researchers compare TSP methods on **benchmark** problems. We use **eil51** from the TSPLIB library: 51 cities, with a published optimal length of **426**.

1. Run the cell that loads eil51. TSPLIB rounds each distance to the nearest whole number, and the notebook does the same, so your numbers match the published ones.
2. Run nearest neighbour on eil51. Write the length and the gap to 426 in section 6 of the report.
3. Brute force on eil51 would try 50! routes, a number with 65 digits. Write one sentence in the report on why that is not possible, using your estimates from Part 6.

> **Screenshot 3 — required**
>
> The nearest-neighbour route on eil51. Paste it into section 6 of the report.

Optional challenge: nearest neighbour depends on where it starts. Run it from every city and keep the best route. What gap can you reach?

Part 8 · The borderline case (20 min)

Ekram receives two new rules from donors:

- **D3, the wedding hall,** can give food only **after 11 pm**, when the wedding ends.
- **D5, the hotel,** can give food only **before 12 noon**.

Discuss with your partner: **is this still the problem you modelled in Part 2?** Can your five parts express these rules? If not, which part must change, and how?

> **A question to settle the argument**
>
> A model is right for a task only if its five parts can express every rule of the task. Look at your state: does it know **what time it is**? Look at your actions: can every action be taken at any time?
>
> Write your answer in section 7 of the report. The same question comes back in Lab 3.

Part 9 · Connect it to the course

Answer these in section 8 of the report. Use the words from the course site.

1. **PEAS.** Think of a route-planning agent that plans Ekram's daily route. Write its PEAS (Module 2, section 2.3). Remember: P describes what counts as success, not how the agent works.
2. **Six properties.** Classify the task environment twice: **as we modelled it** in this lab, and **as it really is** on the streets of Makkah (Module 2, section 2.4). Where do the two columns differ?
3. **Agent design.** Which of the five agent designs in Module 2, section 2.5 fits a route planner that uses brute force? Give one reason.
4. **Be precise.** One sentence on what nearest neighbour guarantees, and what it does not.
5. **From academia to business.** What would Ekram need beyond this lab before using a route planner every day? Name at least three things.

Submit

1. Fill in every section of the report template.

2. Save it as PDF with the name `AI4101_Lab2_<ID1>_<ID2>.pdf`. Replace ID1 and ID2 with your two student IDs.
3. **One** partner submits it, the way your instructor announces in class. Both names and IDs must be on the first page.
4. Keep your Colab notebook. Assignment 2 uses it.

If something goes wrong

Problem	What to try
`NameError: name 'D' is not defined`	A cell above was not run. Use **Runtime → Run all**, or run the cells from the top.
The matrix prints only zeros.	TODO 1 is not finished yet.
The best length is 0.00 km.	TODO 2 is not finished: `route_length` still returns 0.
The notebook says "Complete TODO 1, TODO 2 and TODO 3 first".	Finish the three TODO cells, then run this cell again.
The brute-force timing cell is very slow.	Do not raise the limit above 10 donors. At 12 it takes minutes; at 15, days.
Loading eil51 fails.	Check the internet connection, then run the cell again. The file is read from GitHub.
My changes are gone.	You worked in the original, not in your copy. Use **File → Save a copy in Drive** first.